

Python Common Services Microformat & Message Generator

Table of Contents

Copyright Notice	1
Change History	2
1. Introduction	3
2. Installation	4
3. Code Generation	5
3.1. Command Line Code Generation	5
3.1.1. <code>pys-generate</code> Command	5
3.1.2. Publishing and installation	6
3.1.3. Example Code Generation	7
3.1.4. <code>orca-generate</code> Command	8
3.2. Dynamic Code Generation	9
4. The Generated Microformat Classes	10
4.1. Simple attributes	10
4.2. Enumerate attributes	11
4.3. Entity attributes	11
4.4. Base class attributes	12
4.5. Repeated attributes	13
4.6. Microformat Validation	13
4.6.1. Alternative representations	14
4.7. Microformat versioning	16
4.7.1. Artifact aliases	16
5. MfDoc	17
5.1. Validation	17
6. The Generated Message Classes	18
6.1. Entry Point MicroFormats	18
6.2. Associated Microformats	18
6.3. Message Extensions	19
6.3.1. Aliases	19
6.4. Message Validation	20
6.5. PlantUML Diagrams	21
6.6. Message versioning	21
6.6.1. Messages	21
6.6.2. Extensions	22
6.6.3. Artifact aliases	22
7. Versioning Migration Guide	24
7.1. Legacy package generation	24
7.2. Code migration	24
7.2.1. Import statements	24

7.2.2. Microformat namespacing	25
7.2.3. Message class naming	25

Copyright Notice

© 2019-2020 – CROWN OWNED COPYRIGHT. All Rights Reserved.

The copyright of this Software is vested in the Crown and the Software is the property of the Crown.

This Software is released in confidence, to the recipients nominated by the Crown and is for use only for the purposes of the Crown.

This Software shall not be copied or further disclosed except as authorised by the Crown. Any further disclosure shall be under the terms of this legend and any copies shall be the property of the Crown. On completion of the work in connection with which this Software is released the recipient shall return the Software (and any correspondence) to the issuing authority)

Change History

1.0	Initial Version - Split from pyics-core 4.5.1
1.1	Add mfdoc-to-puml tool
2.0	Add generation of inherited message definitions
2.1	Add generation of extension message definitions
2.2	Add edge inclusion validation checks
2.3	Add vertex inclusion validation checks
2.4	Use networkx to manage microformat trees
3.0	Message/Microformat full validation report
4.0	Allow combined msg/mf packages Include definitions in packages Add split ids for extension messages
4.1	Add ORCA generation support
5.0	Add microformat and message versioning support (See Versioning Migration Guide)

1. Introduction

This package allows generation of Python bindings to Common Services Microformats and Messages.

It is composed of a number of main packages:

- `pycs.generate.mfgen` - the Microformat definition code generator
- `pycs.generate.msggen` - the Message definition code generator
- `pycs.microformats` - the `MfDoc` and `Message` wrapper classes, and utilities used by the generated code

2. Installation

pycs requires an installation of python at least version 3.13. The use of virtual environments is recommended.

```
$ python -m venv venv
$ source ./venv/bin/activate.sh
(venv)$ pip install pycs.microformats
```

To use the pycs package interactively, run python inside the virtual environment

```
(venv)$ python
>>> import pycs.microformats
>>> help(pycs.microformats)
Help on package pycs.microformats:
...
```

3. Code Generation

To assist with creating Microformat and MfDoc XML payloads for Common Services messages, python classes may be generated from microformat definition files. These classes may either be generated into python code files for subsequent import into another application, or generated dynamically into the current application's namespace.

3.1. Command Line Code Generation

Generating python code files allows the generated modules to be installed into the environment for other applications and libraries to use. It is also most suitable when using the definitions from an IDE, as the IDE can then see the full definitions to help with code completion.

3.1.1. **pycs-generate** Command

The **pycs-generate** command is used to generate Microformat and Message classes from their definitions.

```
(venv)$ pycs-generate --help
Usage: pycs-generate [OPTIONS] DEFINITIONS...

Arguments:
  DEFINITIONS...  Definition files to process [required]

Options:
  -p, --package TEXT          the name of the package to create
                              [required]
  -mfm, --mf-module, --microformat-module TEXT
                              name of module to contain the generated
                              python for microformats
  -mm, --msg-module, --message-module TEXT
                              name of parent module to contain the
                              generated python for messages
  -i, --imports TEXT          names of python modules to import containing
                              dependencies
  -r, --requires TEXT         names of modules to add to setup requires
  -o, --output DIRECTORY     Output directory for generated python
                              [default: generated]
  -d, --description TEXT      the package description
  -v, --version TEXT          the version of the package to create
                              [default: 0.0.0.dev0]
  --help                     Show this message and exit.
```

Each invocation of the command generates source code for a new python package as specified by the **--package** option. The definitions are supplied either as individual microformat (**.mfd**), message (**.cm**), collection (**.col**) or extension (**.ext**) files, or as a directory to be searched recursively for matching files.

If the `--microformat-module` option is supplied, the generator will search for any microformat definitions, and generate classes for each of these in an appropriately named module. For example, given a value of `csmf.mf.common`, a microformat named `MsgDistMf` will be available as the class `csmf.mf.common.MsgDistMf`

If the `--message-module` option is supplied, the generator will search for any message, extension and collection definitions, generate classes for each of them, and also include the definition files themselves as resources in the module for use by further generation steps. The module supplied will be used as the base module name, with each definition placed in a sub-module named after the namespace component of the message `TypeId`. For example, given a value of `csmf.msg`, and a message with a `TypeId` of `'urn:csmf:registration:heartbeat'`, generated class will be available as `csmf.msg.registration.Heartbeat`, and its definition file will be available as a resource in the `csmf.msg.registration` module.

If both options are supplied, both microformats and messages will be generated if definitions are found.

If the set of messages or microformats to be generated depend on definitions contained within previously generated packages, these packages must be installed into the virtual environment, and supplied on the command line as `--requires` options. Additionally, the relevant modules within those packages must be specified as `--imports` options.

By default the code will be generated into the `./generated` directory, but this may be changed with the `--output` option.

The generated package will contain a `pyproject.toml` file defining the name, version (as supplied by the `--version` option), any dependencies, and an optional description (as supplied by the `--description` option).

3.1.2. Publishing and installation

The generated packages may be published to a pip repository in the usual way.

```
(venv)$ pip install build twine
(venv)$ python -m build generated --wheel --outdir dist
(venv)$ twine upload dist/*
```

Installation can be done via `pip install` from the repository.

```
(venv)$ pip install my.generated.package
```

Alternatively the package can be installed directly from the generated code, optionally with the `-e` option so that the virtual environment automatically picks up changes to the generated files during development of the definitions.

```
(venv)$ pip install -e generated
```

3.1.3. Example Code Generation

As an example, consider the following directories containing various definitions.

```
common/
|_ mfd/
|_   AMf.mfd
|_   BMf.mfd
|_   CMf.mfd
|_ msg/
|_   collection1.col
app1/
|_ msg/
|_   /subsystem1/message1.cm
|_   /subsystem2/message2.cm
```

If we now wanted to generate a package containing the common definitions we could use the following commands. Assume that the *collection1.col* collection has a `TypeId` of `urn:csmf:common:collection1`.

```
(venv)$ pycs-generate \
  --package example.common \
  --microformat-module example.mf.common \
  --message-module example.msg \
  --output generated/common \
  common
(venv)$ pip install ./generated/common
(venv)$ python
>>> import example.mf.common
>>> help(example.mf.common.AMf)
```

The generated `example.common` package will now contain module `example.mf.common` containing classes `AMf`, `BMf` and `CMf`, and also module `example.msg.common` containing the `collection1.col` definition file. Note that collections do not get a class of their own generated, but are flattened into any messages that use them.

Now assume that the message definitions in the *app1/* directory rely on the microformats and collection definitions from the *common/* directory. To generate code for these definitions we will need to supply the relevant `--requires` and `--import` options to allow the definitions to be found. In this case, there are no microformat definitions, so we do not need to supply `--microformat-module`.

```
(venv)$ pycs-generate \
  --package example.app1 \
  --message-module example.msg \
  --requires example.common \
  --imports example.mf.common \
  --imports example.msg.common \
  --output generated/app1 \
```

```
common
(venv)$ pip install ./generated/app1
(venv)$ python
>>> import example.msg.subsystem1
>>> help(example.msg.subsystem1.Message1)
```

3.1.4. orca-generate Command

Given a repository laid out as an ORCA repository, with artifact files defining which messages and microformats would go into which package, the 'orca-generate' command will generate a package containing bindings for all versions of all available microformat and message definitions, along with convenience modules for each artifact which pull in the definitions required by that artifact.

```
(venv)$ orca-generate --help
Usage: orca-generate [OPTIONS]

Options:
  --artifact-dir DIRECTORY [default: csmf]
  --output DIRECTORY       Output directory for generated python [default:
                           generated]
  --package-name TEXT      package name [default: pycs.csmf]
  --base-module TEXT       base module for the package [default: pycs.csmf]
  --version TEXT           package version
  --dev / --no-dev         generate dev versions [default: no-dev]
  --legacy / --no-legacy   generate legacy packages [default: no-legacy]
  --legacy-base-module TEXT base module for legacy packages [default: csmf]
  --help                   Show this message and exit.

(venv)$ orca-generate
INFO: Generating Microformat module
INFO: Generating Message modules
INFO: Generating Artifact modules

(venv)$ ls generated
pycs.csmf
```

Using the default options, the command will generate a package **pycs.csmf** containing all versions of all microformats, messages and artifacts with the following modules.

pycs.csmf.mf	Contains microformat definitions - See Microformat versioning
pycs.csmf.msg	Contains message definitions - See Message versioning
pycs.csmf.artifact	Contains modules per artifact with imports of microformat and message definitions used

```
(venv)$ orca-generate
INFO: Generating package aoco-123-example-domain/0.0.1.20250520104940
```

3.2. Dynamic Code Generation

Dynamic code generation is most suitable when prototyping at an interactive python prompt, as the definitions can be regenerated quickly, and as the interactive prompt can see the imported definitions they are available for tab completion. It can also be used in an IDE environment, but code completion, syntax highlighting and error checking will not be available as the definitions are not visible to the IDE.

Any Microformats or collections referenced by the messages must have already been imported, either via a previous `create_microformats` call, or by an import of generated code.

```
from pycs.generate.mfgen.generate import create_microformats
from pycs.generate.msggen.generate import create_messages

create_microformats('csmf.mf.common', 'common/mfd/*.mfd', output='generated/common')
create_messages('csmf.msg', 'common/msg/*.col', output='generated/common')
create_messages('csmf.msg', 'app1/msg/*/*.cm', output='generated/app1')
```

4. The Generated Microformat Classes

Each Microformat Entity maps to a python class in the generated module, and each attribute on the entity maps to an attribute on the python class. If any name clashes with a reserved word in python, then its name will have an underscore appended in the generated code. For example, an entity attribute named `continue` would be named `continue_` on the generated class. The root entity in each definition will be a class in the module namespace, and any other entities or enumerates defined inside the microformat will be nested classes inside the root entity.

In the examples in the remainder of this section, the Microformat definitions are taken from modules generated by the ORCA generator from definitions in the ORCA repository.

To set the value of an attribute, the value may be passed as a named parameter to the constructor or assigned directly to an instance of the class. The assignment will make best efforts to convert the supplied value to the correct type. If no conversion to the correct type is possible, an error message is logged, and the original value is left unchanged.

The default value for all attributes on creation of an entity is `None`. This means that if any attribute is marked as `required` the entity will be in an invalid state upon creation, however subsequent attempts to assign `None` to a `required` attribute will result in an error being logged, and the original value left unchanged.

4.1. Simple attributes

`string`, `token` or `uri` values may be supplied as any python value, and its default python string representation will be used. `token` values will have any trailing or leading whitespace removed, and any internal whitespace reduced to a single space character. A `str` will be returned when reading the value.

`bool` values may be supplied as a python `bool` value, as a `str` where any value except `'false'` (case insensitive) and the empty string will evaluate to true, or as any other python value, when the python `bool(v)` function will be used for conversion. A `bool` will be returned when reading the value.

`int32`, `int64` or `real` values may be supplied as `integer` or `float` values, in which case the value supplied will be assigned, with truncation if a `float` is assigned to an integral type, or as a `str`, in which case the string will be parsed to read an appropriate value. An `integer` or `float` will be returned when reading the value.

`timestamp` values may be supplied as a `datetime.datetime` instance, or as an ISO-8601 timestamp formatted `str`, eg `'2019-02-29T12:00:00Z'`. In addition, the empty string `''` or `'now'` will both set the value to the current time. A `datetime.datetime` will be returned when reading the value.

`duration` values may be supplied as either a `datetime.timedelta` instance, as an ISO-8601 duration formatted `str`, eg `'PT30S'` or as an `integer` or `float` value, which will be interpreted as a number of seconds. A `datetime.timedelta` will be returned when reading the value.

Once conversion to the correct type has taken place, any restrictions placed on the value by a `Type` definition are checked. If the supplied value does not comply with these restrictions, then an error

message is logged and the original value is left unchanged.

```
import pycs.csmf

ben_nevis = pycs.csmf.mf.PointOfOriginMf (
    timestamp = 'now',
    latitude = 56.79685,
    longitude = '-5.003508',
    height = 1345 )

snowdon = pycs.csmf.mf.PointOfOriginMf()
snowdon.timestamp = '2019-01-01T12:00:00Z'
snowdon.latitude = '56.068497'
snowdon.longitude = -4.076231
snowdon.height = '1085'
```

4.2. Enumerate attributes

Enumerate values may be supplied as a **str** value containing the name of the enumerate item. For convenience a python class named after the Enumerate is generated with static values named after the item names. These may prove more convenient to use, as an IDE can take advantage of the definitions for code completion and syntax checking. A **str** representation of the item will be returned when reading the value.

```
import pycs.csmf

success = pycs.csmf.mf.OutcomeMf (
    outcome = pycs.csmf.mf.OutcomeEnum.accepted,
    description = 'It worked' )

failure = pycs.csmf.mf.OutcomeMf (
    outcome = 'rejected',
    description = 'No can do' )
```

4.3. Entity attributes

Entity values may be supplied either as an instance of the entity, or as a python **dict** with items corresponding to the attribute names. Additionally, the python ellipsis **...** may be used to initialise an empty instance. Using a dictionary may be useful when dynamically creating instances of unknown type, but in general using the actual type explicitly will give a better experience for code completion and syntax highlighting in an IDE.

```
import pycs.csmf

scafell_pike = pycs.csmf.mf.FocalAreaMf ()
scafell_pike.focalPoint = ...
```

```
scafell_pike.focalPoint.Lat = 54.45422
scafell_pike.focalPoint.Lon = -3.211528

great_gable = pycs.csmf.mf.FocalAreaMf(
    focalPoint = pycs.csmf.mf.FocalPoint (
        Lat = 54.482,
        Lon = -3.219 ))

helvellyn = pycs.csmf.mf.FocalAreaMf(
    focalPoint = {
        'Lat' : 54.57232,
        'Lon' : -3.016054 } )

blencathra = pycs.csmf.mf.FocalAreaMf (
    focalPoint = dict(
        Lat = 54.63985,
        Lon = -3.05046 ))
```

4.4. Base class attributes

Microformat entities are often derived from a base class, for example the Common Services microformat `FocalAreaCircleMf` is derived from `FocalAreaMf`, which is in turn derived from `MfBase`. Any attributes defined on one of the base classes may be accessed directly from the derived entity.

```
import pycs.csmf

scafell_pike = pycs.csmf.mf.FocalAreaCircleMf ()
scafell_pike.focalPoint = ... # FocalAreaMf.focalPoint accessed directly via FocalAreaCircleMf
scafell_pike.focalPoint.Lat = 54.45422
scafell_pike.focalPoint.Lon = -3.211528
scafell_pike.classification = 'U' # MfBase.classification accessed directly via FocalAreaCircleMf
```

However, it is possible for there to be a name clash, with the same attribute defined on both a base class and a derived class. In this case, the name may be qualified by using the base entity name to access the base attribute, with the unqualified version accessing the derived attribute. As there are no name clashes, the following example is equivalent to the preceding one.

```
import pycs.csmf

scafell_pike = pycs.csmf.mf.FocalAreaCircleMf ()
scafell_pike.FocalAreaMf.focalPoint = ... # FocalAreaMf.focalPoint qualified access via FocalAreaMf
scafell_pike.FocalAreaMf.focalPoint.Lat = 54.45422
scafell_pike.FocalAreaMf.focalPoint.Lon = -3.211528
scafell_pike.MfBase.classification = 'U' # MfBase.classification qualified access via MfBase
```

Note that this should almost always be unnecessary, and is only likely if an attribute is added to the base entity in a later that is hidden by an attribute in a derived class that the designer of the base entity does not have visibility of. In all other cases it is recommended to use the form in the first

example above.

4.5. Repeated attributes

An attribute with multiplicity `zeroOrMore` or `oneOrMore` is represented in the python class by a `list`. On creation of the entity the list is empty. Elements may be added or removed from the list using standard python list manipulation operations. If any of the items to be added fail validation as described above, then an error will be logged, none of the items will be added, and the list will remain unchanged from its original value.

```
from pycs.csmf.mf import MfBase

parent = MfBase(
    instId = '6DC86EB7-7725-4F2F-BD6E-3151CBD38A46',
    classification = 'U',
    relation = [
        MfBase.Relation (
            instId = 'A2266CA9-3119-4382-9A83-2355E7E0D44B',
            role = 'child'
        ),
        MfBase.Relation (
            instId = '605D008D-DB73-4721-A789-76562E411CBF',
            role = 'child'
        ),
    ]
)

child_1 = MfBase()
child_1.instId = 'A2266CA9-3119-4382-9A83-2355E7E0D44B'
child_1.relation.append(...)
child_1.relation[0].instId = '6DC86EB7-7725-4F2F-BD6E-3151CBD38A46'
child_1.relation[0].role = 'parent'

child_2 = MfBase()
child_2.instId = '605D008D-DB73-4721-A789-76562E411CBF'
child_2.relation.append(MfBase.Relation(instId = '6DC86EB7-7725-4F2F-BD6E-3151CBD38A46', role = 'parent'))

del parent.relation[0]
```

4.6. Microformat Validation

As mentioned above, when an attribute is assigned a value an attempt is made to convert it to the correct type, e.g. by parsing an integer from a string value. If this is not possible, value of `None` is assigned, and an error logged.

Each microformat definition is generated with a `validate()` function, which will validate whether the data populated on the microformat conforms to any restrictions defined in the definition.

The following checks are done:

- Value has the correct type
- All `required` and `oneOrMore` cardinality constraints are satisfied

- Enumerate values contain one of the allowed values
- String length and pattern constraints are satisfied
- Numeric range constraints are satisfied

If any of the checks fail, the `validate()` function raises a `ValidationError` exception, which contains an `.errors` list of each error found. Each error supplies an `.xpath` showing the path to the error into the XML representation of the microformat, a `.pypath` showing the python code that would be required to reach the error, and a `.text` description of the error. The namespace prefixes used by the XPath can be found at `.nsmap` on the exception.

```
try:
    mf.validate()
except ValidationError as ve:
    for err in ve.errors:
        print(f'Error: {err.pypath}: {err.text}')
```

4.6.1. Alternative representations

Each entity class has a number of methods for producing JSON and XML representations of the entity data.

`entity.to_map()` produces a python `dict` instance containing the entity hierarchy with each attribute of the entity begin an item in the dictionary.

`entity.to_json()` produces a JSON string representation of the entity. Any parameters are passed through to the `json.dumps` method used to format the string. See the `json` module documentation for more details of available parameters.

`entity.to_yaml()` produces a YAML string representation of the entity. Any parameters are passed through to the `yaml.dump` method used to format the string. See the `PyYAML` module documentation for more details of available parameters.

`entity.str()` produces a nicely formatted YAML string representation, suitable for easy human consumption. This method is used internally by python whenever a string representation is needed, eg `print(entity)` or `str(entity)`

`entity.to_xml()` produces an `lxml.etree.Element` node conforming to the schema that can generated by the `microformats` module supplied with Common Services.

```
print(parent) # uses str(parent)
```

produces

```
MfBase:
  classification: U
  instId: 6DC86EB7-7725-4F2F-BD6E-3151CBD38A46
  relation:
```

- role: child
instId: A2266CA9-3119-4382-9A83-2355E7E0D44B
- role: child
instId: 605D008D-DB73-4721-A789-76562E411CBF

```
print(parent.to_json())
```

produces

```
{ "MfBase": { "classification": "U", "instId": "6DC86EB7-7725-4F2F-BD6E-3151CBD38A46", "relation": [{ "role": "child", "instId": "A2266CA9-3119-4382-9A83-2355E7E0D44B"}, { "role": "child", "instId": "605D008D-DB73-4721-A789-76562E411CBF"} ] } }
```

```
print(parent.to_json(indent=4))
```

produces

```
{
  "MfBase": {
    "classification": "U",
    "instId": "6DC86EB7-7725-4F2F-BD6E-3151CBD38A46",
    "relation": [
      {
        "role": "child",
        "instId": "A2266CA9-3119-4382-9A83-2355E7E0D44B"
      },
      {
        "role": "child",
        "instId": "605D008D-DB73-4721-A789-76562E411CBF"
      }
    ]
  }
}
```

```
print(lxml.etree.tostring(parent.to_xml(),encoding=str,pretty_print=True))
```

produces

```
<MfBase:MfBase xmlns:MfBase="microformat://MfBase" classification="U" instId="6DC86EB7-7725-4F2F-BD6E-3151CBD38A46">
  <MfBase:relation role="child" instId="A2266CA9-3119-4382-9A83-2355E7E0D44B"/>
  <MfBase:relation role="child" instId="605D008D-DB73-4721-A789-76562E411CBF"/>
</MfBase:MfBase>print(lxml.etreeimport )
```

4.7. Microformat versioning

When multiple versions of the same microformat are supplied to the code generator, the different versions are available in sub-modules as follows.

The following assumes a microformat named TestMf with definitions for versions 1.0, 1.1, 2.0, 2.1, generated into base module `pys.csmf.mf` as per the convention for the ORCA generator.

class	version	notes
<code>pys.csmf.mf.TestMf</code>	2.1	latest available version
<code>pys.csmf.mf.testmf.TestMf</code>	2.1	latest available version
<code>pys.csmf.mf.testmf.v2.TestMf</code>	2.1	latest minor version for major version 2
<code>pys.csmf.mf.testmf.v2_1.TestMf</code>	2.1	explicit minor version 2.1
<code>pys.csmf.mf.testmf.v2_0.TestMf</code>	2.0	explicit minor version 2.0
<code>pys.csmf.mf.testmf.v1.TestMf</code>	1.1	latest minor version for major version 1
<code>pys.csmf.mf.testmf.v1_1.TestMf</code>	1.1	explicit minor version 1.1
<code>pys.csmf.mf.testmf.v1_0.TestMf</code>	1.0	explicit minor version 1.0

4.7.1. Artifact aliases

For each artifact file supplied to the ORCA generator, a module is generated importing all the microformats referenced by that artifact at the correct major version. The artifact modules are versioned in a similar fashion to microformats above.

When working with microformats used by a particular artifact, it may be that the latest version exposed at the top level of the `pys.csmf.mf` module is not the version required by the artifact in use. In this case it may be more useful to use the definition from the `pys.csmf.artifact.<artifact>` module, which will point to the correct version for that artifact. Each microformat referenced by the artifact will be imported with its original name.

```
import pys.csmf

dist = pys.csmf.artifact.commondomain.MsgDistMf()
abbrev = pys.csmf.artifact.commondomain.AbbreviatedUpdateMf()
```

5. MfDoc

The `pys.MfDoc` class is a representation of the Common Services MfDoc schema, and can be used to produce an XML document suitable for sending to other clients based on Common Services in SILKWAVE messages. The class assumes that any contained microformats have an `instId` attribute, as they will if they have a base of `csmf.mf.common.MfBase`

It has three four main attributes, the `classification` containing SILKWAVE style classification markings, `children` which is a list of child `MfDoc` documents, `microformats` which is a dictionary of contained microformats, indexed by `entity.instId`, and `microformats_by_class` which is a dictionary of contained microformats indexed by the class of the microformat.

To add and remove child MfDocs, simply use the python `list` methods on the `children` attribute.

To add microformats, call the `mfdoc.add_microformats()` method. If any of the microformat does not have an `instId` set the MfDoc will assign a UUID as its `instId`. A microformat may also be added in XML form via this method. If the tag of the XML node is known as a microformat type, then it will be decoded into the relevant class before being add. If the tag is not known, then the node will be added in its raw XML form.

To find a microformat with a specific id, the MfDoc itself acts as a lookup, and `mfdoc[<id>]` will return the requested micorformat. To find all microformats of a particular class, `mfdoc[<class>]` will return a (potentially empty) list of microformats. If the microformat with the specified tag is not known as a microformat, then the raw XML will be returned. If the microformat was not known when the node was added, but has since become known because of a subsequent import statement, then it will be converted into an instance of the microformat class before being returned.

```
from pys import MfDoc
from pys.silkwave import Security

# parent, child_1 and child_2 definitions from above...

mfdoc = MfDoc ( security = Security(classification='U'), parent, child_1, child_2)

mfdoc2 = MfDoc()
mfdoc2.security = Security(classification='U')
mfdoc2.add_microformats(parent, child_1, child_2)

print(mfdoc['6DC86EB7-7725-4F2F-BD6E-3151CBD38A46'])
print(mfdoc[csmf.mf.common.MfBase])
```

5.1. Validation

All the Microformats contained in an `MfDoc` can be validated by calling the `.validate()` function. Each of them will be validated against its definition, as described in [Microformat Validation](#). Additionally, each child `MfDoc` will be similarly validated.

6. The Generated Message Classes

Each Message definition maps to a python class in the generated module. The name of the module will match the namespace of the message id, and the name of the class will match the final component of the message id, with the initial letter converted to upper case.

This message class has a property for each entry-point MicroFormat in the message definition. In the examples in the remainder of this section, the Message definitions are taken from modules generated by the ORCA generator from definitions in the ORCA repository.

6.1. Entry Point MicroFormats

If the cardinality of the microformat is **optional** or **required**, then the property will be a single value of the type of the specified microformat. Any **required** microformats will be pre-populated with an instance.

If cardinality of the microformat is more than one, then the property will be a **Sequence** of the microformat class.

Any properties on the microformat which have fixed values defined in the message definition will have those values automatically set.

```
import pycs.csmf

reg = pycs.csmf.msg.registration.Createsubscriptionrequest()
assert reg.MsgDistMf.type.uri == 'urn:csmf:registration:createsubscriptionrequest'
reg.MsgDistMf.destination.append(...)
reg.MsgDistMf.destination[0].registrantId = 'urn:csmf:registration'
print(reg)
```

6.2. Associated Microformats

Each top-level microformat may have a number of associations to other microformats, and each of these in turn may also have associations to other microformats. These associations are represented as extra properties on the MicroFormat class, and can be used to populate and navigate complex structures of MicroFormats without having to worry about correctly setting the **relation** property in the **MfBase** microformat. If the cardinality of the association is multiple, then the property will be a **Sequence** of the microformat. If any associations are to collections, then these appear as if the collection had been defined inline with the message definition.

```
import pycs.csmf

cap = pycs.csmf.msg.rfsensor.Capability()
cap.SensorCapabilityMf = ...
cap.SensorCapabilityMf.nodeId = 'node-1'
cap.SensorCapabilityMf.SoftwareDesignatorMf.append(...)
```

```
cap.SensorCapabilityMf.SoftwareDesignatorMf[0].version = '1.0'  
cap.SensorCapabilityMf.SoftwareDesignatorMf[0].releaseDate = '2019-10-01T00:00:00Z'
```

6.3. Message Extensions

A sub-module will be created for each message extension, containing a further module for each namespace of the extended messages. This inner module will contain a message class for each message extended. For example if extension `urn:csmf:jelsensor:capability` extends `urn:csmf:rfsensor:capability`, then the extended `Capability` class will be in package `pycs.csmf.msg.jelsensor.capability.rfsensor`, assuming a base module of `pycs.csmf.msg` as per the convention for the ORCA generator. Note that no conversion to CamelCase is performed for the module name.

A message created from an extension message class may be used in place of a message from the original message class, and any microformat extensions will be found as if they had been defined directly on the original message as indicated by the insertion points.

```
import pycs.csmf  
  
cap = pycs.csmf.msg.jelsensor.capability.rfsensor.Capability()  
...  
cap.SensorCapabilityMf.TunerCapabilityMf[0].RfTaskCapabilityMf[0].DifCapabilityMf = ...
```

If an instance of the original message is held and needs to be extended, this can be done by creating a new extension class instance from the original message. A similar technique can be used if multiple extensions are defined on the same base message. All microformats and links between them are retained when converting between message types in this way, but will only be accessible via the high level API when defined on the current message's definition.

```
import pycs.csmf  
  
cap = csmf.msg.rfsensor.Capability()  
...  
extcap = csmf.msg.elint.jelsensor.capability.rfsensor.Capability(mfdoc=cap)  
extcap.SensorCapabilityMf.TunerCapabilityMf[0].RfTaskCapabilityMf[0].DifCapabilityMf = ...
```

6.3.1. Aliases

For convenience, aliases for message extensions are provided in the parent modules, with the submodule names used as prefixes. For the example above, `pycs.csmf.msg.jelsensor.capability.rfsensor.Capability` may also be accessed as `pycs.csmf.msg.jelsensor.capability.RfsensorCapability` and `pycs.csmf.msg.jelsensor.CapabilityRfsensorCapability`

6.4. Message Validation

The generated Message and Microformat classes prevent many errors when creating message instances, by ensuring that any required microformats are pre-populated with a value, setting any attributes with fixed values for that message to the correct value, and populating the correct relation attributes when relationships between Microformats are created. It is still possible, however, to create messages with that do not conform to the specification, by manually changing values, deleting a required microformat, or populating collections of Microformats with too many or insufficient instances.

For this reason a `validate()` method is provided on the generated message class which will validate the following:

- Correct cardinality of associations
- Correct value of fixed attributes
- Only one instance of mutually exclusive Microformats present (for generalization/specialization)
- Vertex and Edge constraints
- Each microformat in the tree is valid according to its `validate()` method
- Dangling relationships, that is `relation` instances that have no corresponding Microformat in the message.
- Any child MfDocs are valid. If the child `MfDoc` is an explicit message type, then full checking is done, otherwise only the basic `MfDoc` checks are performed.

An `extension_selector` argument may supplied, which should be a function taking the message as a parameter and returning a list of extensions as an id and version tuple. The message will then also be validated against the extension definitions returned. If no selector is supplied, only the base message will be validated.

For example, in Common Services, the `MsgDistMf` microformat could be queried as follows:

```
def extract_extensions(msg: MfDoc) -> list[tuple[str,str]]:
    if not MsgDistMf in msg:
        return []

    return [(e.id, i.version) for e in msg[MsgDistMf][0].extensions ]

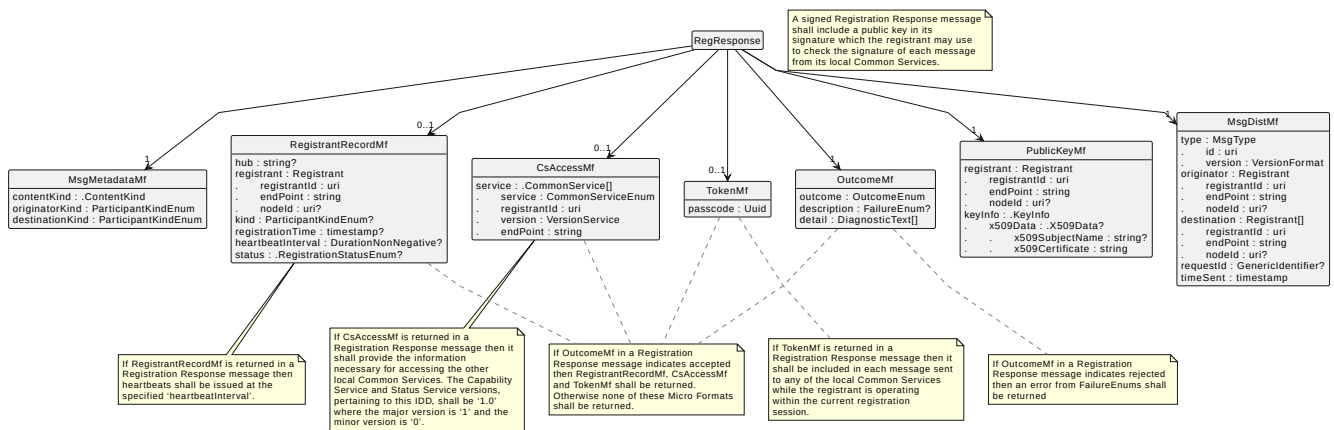
msg.validate(extension_selector = extract_extensions)
```

A `ValidationError` exception will be raised if any errors are found. See [Microformat Validation](#) for further details of this exception.

Note that this will also perform any validation on Microformats which are not navigable to via the generated Message class, e.g. if extension Microformats have been added to the underlying `MfDoc` either programmatically or by parsing an XML document with extra Microformats present. This validation for non-message Microformats will only validate against the basic Microformat checks.

6.5. PlantUML Diagrams

As part of the message generation, each Message class is provided with a `puml` attribute which contains a string with a PlantUML class diagram of the Microformat hierarchy. These diagrams show the relationships between different microformats in the message, and the structure of the attributes within each microformat, along with any notes from the definitions. Any attributes inherited from `MfBase` are omitted for clarity. When using an IDE which supports PlantUML (eg PyCharm with the PlantUML plugin), these diagrams can be displayed alongside the code to assist with navigating and populating the messages.



Additionally, an `AsciiDoc` document for each module is generated in the output directory containing PlantUML diagrams for each message in that document. AsciiDoc can then be used to turn this into HTML, PDF or other formats as required.

WARNING

As the PlantUML code embedded in the class definitions is statically generated so that it can be seen by the IDE, it reflects the attributes present on the Microformats at the time of generation. If the microformat definitions are sourced from a different package and microformats regenerated, the message definitions will also need regenerating to keep the UML diagrams up to date.

6.6. Message versioning

6.6.1. Messages

When multiple versions of the same microformat are supplied to the code generator, the different versions are available in sub-modules as follows.

The following assumes a message with urn `urn:csmf:example:testmsg` with definitions for versions `1.0`, `1.1`, `2.0`, `2.1`, generated into base module `pycs.csmf.msg` as per the convention for the ORCA generator.

class	version	notes
<code>pycs.csmf.msg.example.Testmsg</code>	2.1	latest available version
<code>pycs.csmf.msg.example.testmsg.Testmsg</code>	2.1	latest available version
<code>pycs.csmf.msg.example.testmsg.v2.Testmsg</code>	2.1	latest minor version for major version 2

class	version	notes
<code>pycs.csmf.msg.example.testmsg.v2_1.Testmsg</code>	2.1	explicit minor version 2.1
<code>pycs.csmf.msg.example.testmsg.v2_0.Testmsg</code>	2.0	explicit minor version 2.0
<code>pycs.csmf.msg.example.testmsg.v1.Testmsg</code>	1.1	latest minor version for major version 1
<code>pycs.csmf.msg.example.testmsg.v1_1.Testmsg</code>	1.1	explicit minor version 1.1
<code>pycs.csmf.msg.example.testmsg.v1_0.Testmsg</code>	1.0	explicit minor version 1.0

6.6.2. Extensions

An extension may extend multiple different versions of the same message, so a similar module structure is used for both the extension module and the submodules containing the extended messages.

The following assumes an extension 'urn:csmf:extensions:testtext' version **1.0** extending a message `urn:csmf:example:testmsg` versions **1.0**, **1.1**, **2.0**, **2.1**, generated into base module `pycs.csmf.msg` as per the convention for the ORCA generator.

class	extension version	message version	notes
<code>pycs.csmf.msg.extensions.testtext.example.Testmsg</code>	1.0	2.1	latest available version
<code>pycs.csmf.msg.extensions.testtext.example.testmsg.Testmsg</code>	1.0	2.1	latest available version
<code>pycs.csmf.msg.extensions.testtext.example.testmsg.v2.Testmsg</code>	1.0	2.1	latest minor version for major version 2
<code>pycs.csmf.msg.extensions.testtext.example.testmsg.v2_1.Testmsg</code>	1.0	2.1	explicit minor version 2.1
<code>pycs.csmf.msg.extensions.testtext.example.testmsg.v2_0.Testmsg</code>	1.0	2.0	explicit minor version 2.0
<code>pycs.csmf.msg.extensions.testtext.example.testmsg.v1.Testmsg</code>	1.0	1.1	latest minor version for major version 1
<code>pycs.csmf.msg.extensions.testtext.example.testmsg.v1_1.Testmsg</code>	1.0	1.1	explicit minor version 1.1
<code>pycs.csmf.msg.extensions.testtext.example.testmsg.v1_0.Testmsg</code>	1.0	1.0	explicit minor version 1.0

If multiple versions of the extension definition were available, they would use a similar module structure to the table in the previous section, with the extended messages relevant for that version following the structure above.

6.6.3. Artifact aliases

For each artifact file supplied to the ORCA generator, a module is generated importing all the messages referenced by that artifact at the correct major version. The artifact modules are versioned in a similar fashion to messages above.

When working with messages used by a particular artifact, it may be that the latest version exposed at the top level of the `pys.csmf.msg.<namespace>` module is not the version required by the artifact in use. In this case it may be more useful to use the definition from the `pys.csmf.artifact.<artifact>` module, which will point to the correct version for that artifact. Each message referenced by the artifact will be imported with its original name prefixed with its namespace to avoid ambiguity.

```
import pys.csmf

subscribe_status = pys.csmf.artifact.commonservicesmessages.StatusCreatesubscriptionrequest()
subscribe_capability = pys.csmf.artifact.commonservicesmessages.CapabilityCreatesubscriptionrequest()
```

7. Versioning Migration Guide

Version 5.0 of the code generator introduced support for multiple versions of the same microformat or message. This necessitated changes to the module and package structure, and introduced incompatibilities with packages generated with the old version.

7.1. Legacy package generation

To assist with migration, the [ORCA generator](#) has the ability to generate packages with the same layout as prior versions which pull in the relevant definitions from the `pycs.csmf` package. For the latest version of each artifact file, a package is generated `aoco-<aoco>-<name>`, where the name is taken from the artifact file name, and the AOCO number taken from that defined in the file. Any microformats will be defined in the module `csmf.mf.<sname>` where `<sname>` is the package name with any `-domain` or `-message` suffix removed, and any hyphens removed. Any messages will be defined in the module `csmf.msg.<sname>.<module>` where `<module>` is the namespace portion of the URN in the message definition.

```
(venv)$ orca-generate --legacy
INFO: Generating Microformat module
INFO: Generating Message modules
INFO: Generating Artifact modules
INFO: Generating legacy compatibility packages

(venv)$ ls generated
aoco-533-common-domain
aoco-534-cs-domain
aoco-535-geolocation-domain
aoco-536-observable-domain
aoco-537-rf-sensor-domain
aoco-539-common-services-messages
aoco-540-rfextgeo-messages
:
pycs.csmf
```

7.2. Code migration

Using the legacy packages above should allow existing client code to be used unchanged, however they only allow access to the message and microformat versions defined in the artifact, and do not allow access to other versions. Migrating code to use the single `pycs.csmf` package also brings a number of other advantages.

7.2.1. Import statements

Previously, each message namespace that was expected to be used had to be imported separately to register the codecs with the `pycs.client` library to allow its messages to be received and decoded. With the `pycs.csmf` library, a single `import pycs.csmf` statement will register all microformats and

messages, and allow module-qualified access within code.

Legacy	<code>pys.csmf</code>
<pre>import csmf.msg.registration import csmf.msg.capability import csmf.msg.status import csmf.mf.common</pre>	<pre>import pys.csmf</pre>

7.2.2. Microformat namespacing

All microformats are now defined within a single module `pys.csmf.mf`, removing the need to remember which artifact namespace they are defined in.

Legacy	<code>pys.csmf</code>
<pre>import csmf.mf.common import csmf.mf.cs dist = csmf.mf.common.MsgDistMf() abbrev = csmf.mf.cs.AbbreviatedUpdateMf()</pre>	<pre>import pys.csmf dist = pys.csmf.mf.MsgDistMf() abbrev = pys.csmf.mf.AbbreviatedUpdateMf()</pre>

7.2.3. Message class naming

Older versions of the code generator employed an algorithm to try and chop message names up into individual words (based on a dictionary of common words found in message names), and then capitalise these names using CamelCase. For example, `urn:csmf:capability:cancelsubscriptionrequest` would result in a class name of `CancelSubscriptionRequest`. This approach has proved unreliable and unmaintainable as the scope of subjects covered by messages has increased. A much simpler approach has now been adopted whereby only the first letter of the name from the urn is capitalised to form the class name. Note that any capitalisation in the urn will be maintained, but all existing message definitions follow the convention of fully lowercase names.

Legacy	<code>pys.csmf</code>
<pre>import csmf.msg.capability subscribe = csmf.msg.capability .CreateSubscriptionRequest()</pre>	<pre>import pys.csmf subscribe = pys.csmf.msg.capability .Createsubscriptionrequest()</pre>